

Unsupervised learning.

Structure without labels.

Group similar movies. Keep fewer coordinates.

Understand what the discovered patterns tell you.

🕒 16-20 min core reading · 20-30 min practice · 12 short sections

Sometimes we have descriptions of movies but no viewer ratings to predict. We can still look for groups, compress a representation, or flag unusual records. The key is to understand what the discovered structure actually tells us.

WHAT YOU WILL UNDERSTAND

Two practical tools: k-means groups nearby movies; PCA stores a smaller representation while discarding some detail.

The wider map: distinguish supervised, self-supervised, semi-supervised, and reinforcement learning by where their learning signal comes from.

Reading path: short core explanations, three diagrams, a working movie explorer, and a Python workbook. Open the deeper notes when you want the extra detail.

1. Look for structure without supplied outcome labels

In Chapter 13, every training event had an outcome: a rating or a thumbs-up. **Unsupervised learning** instead looks for structure in inputs without being given the desired output label for each example. There is still an objective, such as making groups compact or reducing reconstruction error.

Imagine arranging a movie catalog into shelves before collecting viewer feedback. **Clustering** groups movies under a chosen notion of similarity. **Dimensionality reduction** represents movies with fewer coordinates.

Anomaly detection scores how unusual a record is under some learned pattern.

Our new, synthetic snapshot contains nine fictional movies with two invented scores: **action** and **humor**, each from 0 to 10. These are authored descriptions, not objective measurements or actual audience feedback. No genre, cluster, or rating target is supplied. The input features can involve human judgment even when the learning task is unsupervised.

2. Decide what “similar” means

Represent a movie as a point $x = (\text{action}, \text{humor})$. Moonlight Map is $(2, 3)$; Forest Signal is $(3, 4)$. Their squared Euclidean distance is:

$$d^2 = (2 - 3)^2 + (3 - 4)^2 = 2.$$

Their ordinary distance is $\sqrt{2}$. Movies are “close” here because of these two scores; language, runtime, plot, and a viewer’s taste are absent from the representation.

If we multiply the humor coordinate by 3, the same squared distance becomes $1 + 9 = 10$. Its humor contribution grows **ninefold** because the changed difference is squared. Feature choice, scale, and weighting can change the groups you discover.

▼ Go deeper: scaling is part of the definition

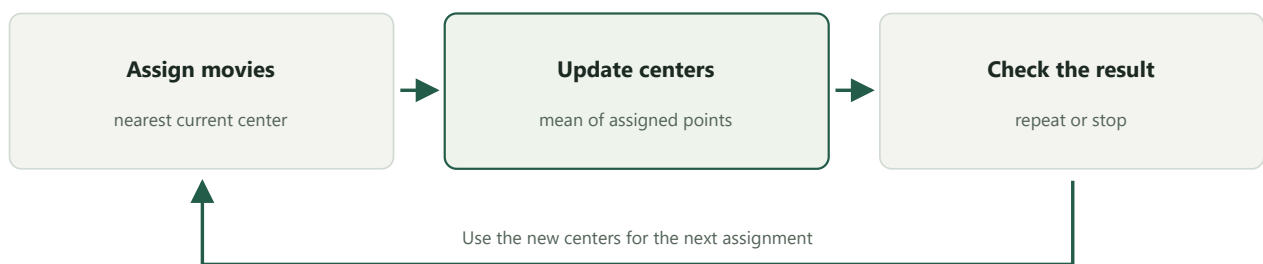
A runtime measured in minutes may dominate a 0-1 preference feature unless the representation handles their scales. Standardization can help, but equal variance is a modeling choice, not a guarantee that features deserve equal importance. Here both scores already use 0-10; the $\times 3$ control deliberately changes their relative influence.

Euclidean distance is one option. Other representations may call for cosine similarity, categorical distances, or a learned metric. Whatever you choose must fit the method’s assumptions: ordinary k-means uses squared Euclidean distances and arithmetic means. Do not substitute an arbitrary distance while assuming the same update is still valid.

3. Follow the k-means loop

Choose a number of groups k and an initial center for each. Assign each movie to its nearest center. Then move every center to the **mean** of its assigned points and assign movies again. Repeat until a stopping condition is met. The center is a **centroid**; it need not coincide with an actual film.

➤ K-means alternates between groups and their means



An objective exists even without supplied target labels.

Figure 1. Choose starting centers, assign each movie to its nearest center, then move centers to group means. Repeat until the chosen stopping rule is met. Cluster IDs have no built-in genre meaning.

The objective, often called **inertia**, is the sum of squared distances from points to their assigned centers:

$$J = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2.$$

Here n counts movies, c_i identifies movie i 's assigned cluster, and μ_{c_i} is that cluster's center. The assignment and mean updates each avoid increasing this objective, apart from numerical rounding. See [scikit-learn's k-means reference](#).

Our default uses $k = 3$ and starts at Moonlight Map, Cafe Chaos, and Funny Mission. The first group contains six movies whose action scores sum to 23 and humor scores sum to 14. Its updated centroid is $(23/6, 14/6) \approx (3.8333, 2.3333)$. One update lowers inertia from **75 to 51.1667** and leaves the assignments unchanged.

▼ Go deeper: initialization, stopping, and empty groups

"Spread start" in this workbook means fixed first, middle, and last rows for $k = 3$; for $k = 2$ it uses first and last. "First k " uses the first rows. These are reproducible teaching choices, not the probabilistic k-means++ initialization used by many libraries. Row order can therefore affect the starting points.

Step 0 already includes nearest-center assignments. Each button press updates means and then reassigns movies. We stop when those assignments no longer change, with a maximum of 50 updates. At an intermediate step, the centers are means of the *previous* groups; another update may be needed.

Distance ties go to the lowest center index. An empty group retains its old center. Duplicate coordinates can therefore leave fewer occupied groups than the requested k . Cluster numbers are arbitrary identifiers; exchanging C1 and C2 does not change a partition.

4. Ask whether the groups are useful

A lower inertia means a tighter fit under the **same data and coordinates**. It does not establish correct genres or better recommendations. Our two starts with $k = 3$ stop at 51.1667 and 55.3333: different locally stable solutions to the same problem. Trying multiple starts helps, but a stopping point is not proof of the best possible solution.

Choosing k is also a judgment. The best achievable inertia cannot increase when more centers are allowed, so minimizing it alone favors ever more groups. Individual runs may behave differently because of initialization. An **elbow plot** looks for diminishing improvement; a **silhouette score** compares within-group closeness with separation from another group. Neither identifies universally “true” clusters.

Inspect movie members, check sensitivity to reasonable changes, and ask whether the grouping helps the intended task. Two features cannot reveal every meaningful pattern. K-means is especially limited when groups have strongly different shapes or densities; Google’s [k-means limitations](#) explain these tradeoffs.

▼ Go deeper: other clustering families

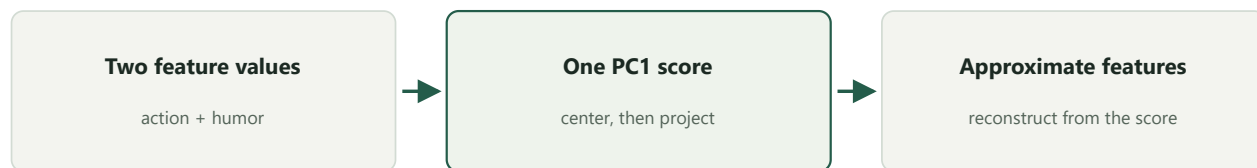
Hierarchical clustering builds nested groups, often visualized as a dendrogram; choosing where to cut the hierarchy sets a grouping. **DBSCAN** groups density-connected points and can mark sparse points as noise, with neighborhood and density settings controlling the result. It does not require the same fixed k choice as k-means.

A **Gaussian mixture model** fits a mixture of probability distributions and can give a movie soft membership probabilities instead of only one group ID. Assumptions still matter: a soft assignment is not a viewer’s liking probability. These methods answer related questions with different models; they are not implemented in this workbook. See the [primary clustering guide](#) and [mixture-model guide](#).

5. Keep fewer coordinates with PCA

Principal component analysis (PCA) centers the feature vectors, then finds perpendicular directions that capture successively smaller amounts of variance. Keeping only the first principal component, **PC1**, gives each movie one coordinate along the direction of greatest variance.

➤ One coordinate can retain structure and lose detail



Keep variation along the line; discard the sideways difference.

High retained variance is not evidence of high prediction accuracy.

Figure 2. PCA learns directions from centered feature values. Keeping one direction stores one score per movie. Reconstructing two coordinates from that score generally loses the perpendicular detail.

With mean vector μ and unit direction v , the stored score and reconstructed point are:

$$z = (x - \mu) \cdot v, \quad \hat{x} = \mu + zv.$$

This connects to Chapter 6's dot product and projection. For a hand calculation, take three points $(1, 1)$, $(2, 2)$, $(3, 3)$. Their mean is $(2, 2)$, and the principal direction is $(1, 1)/\sqrt{2}$. The point $(3, 3)$ has score $z = \sqrt{2}$ and reconstructs exactly. All three points already lie on one line, so this example loses no detail.

Our nine movies do not lie on one line. Their PC1 retains **69.97% of total feature variance**. Cafe Chaos, $(2, 9)$, reconstructs to about $(5.6886, 5.5312)$: its low-action, high-humor character gets blurred. The retained percentage describes the representation, **not recommendation accuracy**. See the [PCA guide](#).

▼ Go deeper: eigenvectors, reconstruction, and visual maps

The covariance matrix summarizes how centered features vary together. Its eigenvectors provide PCA directions; their eigenvalues give variance along those directions. Our small implementation solves the two-dimensional case directly and divides covariance sums by n . Using $n - 1$ rescales the eigenvalues but leaves these directions and explained-variance ratios unchanged.

The movie mean is about $(4.6667, 4.4444)$, and PC1's unit direction is $(0.6851, 0.7285)$. Reversing the direction reverses every score but reconstructs the same points. With two retained components, these two-dimensional points can be reconstructed fully; with one, the mean squared reconstruction distance is 5.7459. If both features are constant, the variance ratio is undefined and the workbook stops with an error.

PCA creates combinations of features, rather than selecting one original feature. It is sensitive to feature scales and does not use a target to decide which variance matters. Nonlinear visualizations such as t-SNE or UMAP serve different goals; an apparent island in a two-dimensional map is not, by itself, evidence of a real category.

Autoencoders learn another kind of compressed representation; we will meet them with neural networks.

6. Treat unusual as a reason to inspect

Anomaly detection flags records that differ from a reference pattern. A catalog entry might be unusual because of an input error, a rare valid movie, or features that fail to describe it well. Unusual does not mean invalid or unpopular.

For example, Cafe Chaos has the largest squared distance from its PCA reconstruction in our snapshot. That means it sits far from this particular one-dimensional representation. A distance from a cluster center would be a different score with different assumptions. Neither alone justifies removing the film.

Outlier detection examines potentially unusual examples within the data being analyzed. **Novelty detection** typically learns a reference from data assumed to represent normal cases, then judges new examples. See [scikit-learn's outlier and novelty guide](#).

This chapter fits the entire small collection to describe it. It makes no claim about held-out accuracy. If PCA, scaling, or clustering becomes a preprocessing stage for a future prediction task, fit that stage only on the appropriate training split and transform held-out examples using the saved fit. Unsupervised preprocessing can still leak information across Chapter 12's boundary.

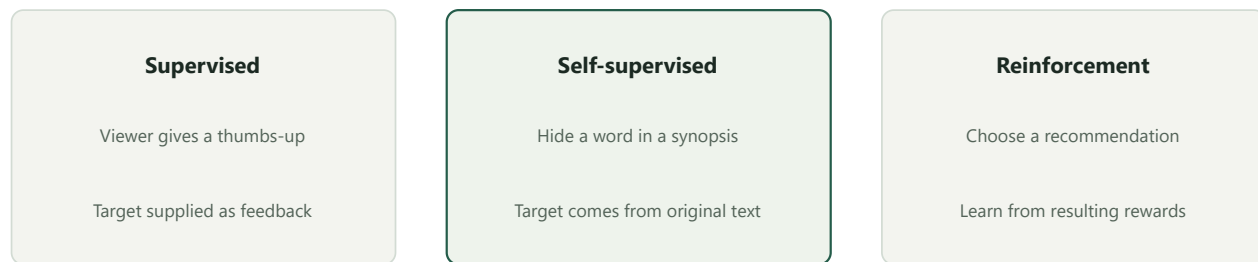
7. Find out where the learning signal comes from

"No manually labeled target for every row" covers several different situations. Keep these three distinctions clear:

Approach	Movie example	Where the target or signal comes from
Self-supervised	Hide a word in a synopsis and predict it	The original text supplies the missing word
Semi-supervised	Learn a genre classifier using a few labeled synopses and many unlabeled ones	A combination of supplied labels and unlabeled structure
Weak supervision	Use imperfect rules to suggest genre labels	Noisy heuristics or other indirect signals

Self-supervised learning still has targets and a loss. Masked-word prediction is one example; predicting a next token is another. Such training can produce representations later adapted to labeled tasks. The [BERT paper](#) demonstrates masked-language pretraining; [Data Programming](#) studies learning with noisy labeling functions.

➤ Ask where the learning signal comes from



These describe learning signals, not interchangeable names for algorithms.

Figure 3. Human feedback, targets derived from raw text, and rewards from interaction can train different systems. Having fewer human annotations does not mean having no training objective.

▼ Go deeper: pseudo-labels can reinforce mistakes

One semi-supervised method first trains on labeled examples, predicts labels for unlabeled ones, and adds selected predictions to training. These are **pseudo-labels**, not newly verified facts. A model confident in the wrong pattern can amplify its own mistakes. Useful unlabeled data and reliable assumptions about the data distribution matter; more rows do not guarantee improvement.

A rule saying “a synopsis containing ‘spaceship’ means science fiction” is weak supervision: it supplies an imperfect category guess. Hiding the actual word “spaceship” and recovering it is self-supervision: the original word is the target. These categories can overlap in a larger workflow, but their signals should remain explicit. See scikit-learn’s semi-supervised learning guide.

8. Separate feedback, label requests, and update schedules

In **reinforcement learning (RL)**, a policy chooses actions using available state information, receives rewards from interaction, and is learned to improve expected cumulative reward. A recommendation system could choose a film to show, observe feedback, and learn a policy over repeated interactions. Reward design matters: a click may not represent long-term satisfaction. Merely collecting clicks does not make a system RL; those clicks could also become ordinary supervised labels.

Active learning chooses which examples to ask someone to label—for example, requesting genre judgments for confusing synopses. **Online learning** updates a model as data arrives instead of only fitting one fixed batch. **Transfer learning** reuses knowledge learned on another task or dataset. These describe different aspects of learning and can be combined.

Google’s [machine-learning glossary](#) defines these terms. Chapter 38 develops RL further. This chapter’s executable experiment implements k-means and PCA; it does not train a language model or an RL policy.

9. Explore groups and compressed coordinates

Start with $k = 3$, the spread start, and humor $\times 1$. Move the centers once: confirm **75** $\rightarrow$ **51.1667** and inspect the group members. Switch to the first-row start and run again. Then choose $k = 2$ and compare humor $\times 1$ with $\times 3$: Cafe Chaos changes its companions.

Switch to PCA and inspect Cafe Chaos’s dashed reconstruction difference. The chart always uses original score units. PCA uses the original unweighted scores, independent of the clustering controls.

DISCOVER GROUPS. COMPRESS A REPRESENTATION.

Real calculations · fictional movies

Same movies. Different structures.

Nine movies have invented action and humor scores from 0 to 10. There are no supplied cluster labels or viewer-rating targets. Both methods fit this whole small collection for exploration.

Default initialization: $k = 3$, spread start, original scores. Initial centers are M001, M005, and M009. Initial inertia is 75. After one update, assignments stabilize and inertia is 51.1667.

k	Start	Humor scale	Final inertia
3	spread	$\times 1$	51.1667
3	first	$\times 1$	55.3333
2	spread	$\times 1$	92.1429
2	spread	$\times 3$	143.5000

Only compare inertia when feature coordinates and data are the same. The two $k = 3$ starts reach different local solutions. Requested groups have no supplied “correct genre” labels.

PCA on the unweighted scores retains 69.97% of total variance in one component. Its mean squared reconstruction distance is 5.7459. Cafe Chaos (2, 9) reconstructs to about (5.6886, 5.5312), showing a meaningful difference that compression discards.

Compare inertia only when the data and feature coordinates match. Multiplying humor by 3 changes the objective’s scale as well as the grouping. A larger number across these differently scaled experiments is not evidence that the new grouping is worse.

10. Run the workbook and practice

Extract its four files together. Python 3.10 or newer is sufficient; there are no third-party packages or API calls. Open a terminal in that folder:

```
py unsupervised.py --steps 0
py unsupervised.py --steps 1
py unsupervised.py --start first
py unsupervised.py --k 2 --humor-weight 3
py unsupervised.py --mode pca
py -m unittest -v test_unsupervised.py
```

Use `python` or `python3` if that is your working command. By default, the script reads `movie_features.json` beside itself, runs clustering until the stopping rule, and prints JSON. It leaves the input unchanged. `--steps 0` reports initialization; PCA prints the learned direction, scores, reconstructions, and residuals. Invalid input produces an error and exit code 2.

Exercise 1: change the geometry

For Moonlight Map (2, 3) and Forest Signal (3, 4), calculate squared distance before and after multiplying humor by 3. Did the movie descriptions change?

▼ Reveal the reasoning

Originally, $1^2 + 1^2 = 2$. With the second coordinate scaled, $1^2 + 3^2 = 10$. The stored descriptions did not change; the representation used to measure similarity changed. The humor contribution increased from 1 to 9.

Exercise 2: explain the fitted center

The default first cluster has six members, with action sum 23 and humor sum 14. Find its updated centroid. Does its lower inertia establish the correct genre for those six films?

▼ Reveal the reasoning

Divide each sum by six: the centroid is (23/6, 14/6), or about (3.8333, 2.3333). Run the default one-step workbook to verify it and inertia 51.1667. The objective measures compactness under our chosen scores; no supplied genre labels or viewer outcomes establish a correct interpretation.

Exercise 3: project and reconstruct

For the three collinear points in Section 5, use mean (2, 2) and direction $(1, 1)/\sqrt{2}$ to project (3, 3) and reconstruct it. Why is the movie example less exact?

▼ Reveal the reasoning

Centering gives (1,1); its dot product with the unit direction is $2/\sqrt{2} = \sqrt{2}$. Reconstruction is $(2,2) + \sqrt{2} \times (1,1)/\sqrt{2} = (3,3)$. These three points lie on the retained line, so no detail is lost. The nine movie points also vary perpendicular to their fitted line; storing just one coordinate discards that variation.

11. Check your understanding

01 K-means places two movies in the same cluster. What does that establish?

- A. Every viewer will enjoy both films.
- B. They were grouped under the chosen features, distance, and fitting setup.
- C. They must share the same real-world genre.

Answer: B. Clusters describe structure under a representation and objective. Their IDs and meanings are not supplied truth labels or promises of viewer preference.

02 The humor coordinate is multiplied by 3. What happens to its squared-distance contribution?

- A. It is multiplied by 9.
- B. It is divided by 3.
- C. It cannot affect clustering.

Answer: A. The difference is multiplied by 3 before squaring, so its contribution becomes 9 times as large. This also changes the units of the clustering objective.

03 PCA retains 69.97% of feature variance. What can you conclude?

- A. It predicts 69.97% of movie ratings correctly.
- B. The discarded information is always irrelevant.
- C. That fraction of variance is preserved by this linear projection; usefulness needs separate evaluation.

Answer: C. Explained variance describes representation, not target accuracy. A low-variance feature may still matter for a downstream task.

04 A model hides a word in a synopsis and learns to recover it. Which signal is used?

- A. A human provides a new category for every masked example.
- B. A self-supervised target comes from the original data.
- C. There is no training target or loss.

Answer: B. Self-supervision constructs prediction targets from the data itself. It still uses an objective and can later support supervised applications.

05 An unusual movie receives a large anomaly score. What should happen next?

- A. Delete it as certainly invalid.
- B. Treat the score as proof that nobody will like it.
- C. Inspect the record and the scoring assumptions before making a decision.

Answer: C. Unusual can mean an error, a rare valid film, or a poor representation. A score alone does not establish the cause or justify a product action.

12. Connect the objective to the claim

You can now trace **features** → **similarity or variance objective** → **fitted structure** → **interpretation**. A cluster is not a genre fact; a PCA coordinate is not a rating; an anomaly score is not a verdict.

Before choosing a method, ask what data is available, where the learning signal comes from, and how usefulness will be checked. Discovered groups may help organize a catalog or create candidate recommendations, but Chapter 11's runtime and catalog constraints still need explicit checks.

▼ Quick-reference glossary

Term	Meaning here
Unsupervised learning	Learn input structure without supplied outcome labels
Cluster / centroid	A group / its mean feature vector in k-means
Inertia	Sum of squared distances to assigned centers
Initialization	Starting parameters before iterative fitting
PCA / principal component	Linear dimensionality reduction / a learned variance direction
Explained variance ratio	Fraction of total variance retained by selected components
Reconstruction residual	Difference between an original and reconstructed point
Anomaly	A record unusual under a specified reference pattern
Self-supervision	Construct learning targets from the original data
Semi-supervision / pseudo-label	Use labeled and unlabeled data / a model-generated target
Weak supervision	Learn with noisy or indirect labeling signals
Policy / reward	Rule for choosing actions / feedback optimized in RL

Primary references: scikit-learn's [clustering](#), [PCA](#), [anomaly detection](#), and [semi-supervised learning](#) guides; Google's [ML glossary](#); the [BERT](#) and [Data Programming](#) papers. Examples, diagrams, and calculations use our original synthetic movie snapshot.

Next: Chapter 15 - Evaluating and improving ML models. Choose useful metrics, compare models fairly, and diagnose the errors that matter.